Guaranteed Tail Calls in C++

Document #:	Unnumbered draft, R0
Date:	2026-10-09
Project:	Programming Language C++
Audience:	EWG

Authors:	Ian Pike
	<ianpike98cs@gmail.com>

Intent: Proposal for design feedback

Abstract

This paper proposes `[[always_tail]]` for checked tail transfers that end the caller's activation, with diagnosed rejection when unsupported. Protobuf's C++ parser and three compiler families establish use and implementation experience. A test across separate translation units on Apple Clang 21, arm64 macOS, shows ordinary dispatch exhausting the stack without optimization while `clang::musttail` and a trampoline succeed. No speed advantage is claimed. This supports developing a common contract, while a portable minimum domain, mandatory attribute semantics, and WG21's implementation-limit objection remain unresolved.

1 Revision History

R0: Initial working draft. No paper number has been assigned. Standard wording remains to be developed.

2 Introduction

Protobuf's C++ parser chains field handlers through a dispatch table. Its source explains that this chain needs proper tail calls to avoid exhausting the stack. When the required facility is unavailable, it returns through an enclosing parse loop instead. [ProtobufTailCalls] This is an existing library maintaining two dispatch paths because ordinary C++ calls do not carry the required guarantee.

This paper proposes `[[always_tail]]` on a return statement. An accepted transfer ends the caller's activation before entering the callee, which returns to the caller's caller. An unsupported transfer is diagnosed. The implementation cannot silently retain a chain of caller activations because optimization was disabled or failed.

The justification has three parts: a maintained C++ use, implementations in Clang, GCC, and MSVC, and a reproducible comparison with ordinary calls and a trampoline. These establish a useful mechanism. They do not establish support for this particular standardization design, a portable minimum domain, or a speed advantage.

3 The failure depends on input length

An interpreter can put each operation in a separate function that selects and calls the next operation. A parser can do the same for fields. A loop in the interpreted or generated program then becomes a cycle of calls in the implementation. The number of transfers depends on the input. Keeping one activation per transfer turns an iterative computation into an unbounded use of call-stack storage.

Ordinary `return f(args...)` does not require elimination of the caller's activation, [stmt.return]. Testing a release executable therefore cannot establish that the same dispatch design remains bounded in a debug build. Increasing the stack limit only moves the input length at which an accumulating chain fails.

4 Protobuf uses the guarantee and maintains a fallback

In the pinned Protobuf source, `TagDispatch` calls the target selected from the field table through a function pointer. `ToTagDispatch` chooses between continuing that chain and returning to `ParseLoop`. Its `always_return = !PROTOBUF_TAILCALL` condition selects the latter when guaranteed tail calls are unavailable. [ProtobufTailCalls]

The fallback matters to the argument. Bounded parsing is already expressible in ordinary C++. The missing facility is a checked transfer between separately defined functions without retaining the caller. Protobuf is evidence that a real C++ library chooses this representation when it can rely on the guarantee. It is not evidence that a trampoline cannot work or must be slower.

Function-pointer calls must be part of a useful design. A facility limited to direct self-recursion would not cover this dispatch pattern. The full Protobuf signature, calling convention, and cleanup requirements still need to be mapped to the proposed domain before claiming that the standard facility could replace its extension.

5 A reduced experiment reproduces the failure

The local experiment places two handlers and the dispatch table in separate translation units. Each handler changes unsigned state and selects the other handler through a function pointer. The state belongs to the enclosing driver, outside the handler activations. One handler, using the proposed spelling, is:

```
#include <cstdint>

struct State {
    std::uint64_t remaining;
    std::uint64_t value;
};

using Operation = std::uint64_t (*)(State*);
extern Operation operations[2];

std::uint64_t add(State* state) {
    if (state->remaining == 0) return state->value;
    state->value += 0x9e3779b97f4a7c15ULL;
    --state->remaining;
    [[always_tail]] return operations[1](state);
}
```

Three variants perform the same 10 million state updates: ordinary calls, required tail calls, and a trampoline whose handlers return the next operation to a loop. The tested attribute is `[[clang::musttail]]`. The proposed spelling is not implemented.

Dispatch	Optimization disabled	Optimization enabled
Ordinary calls	Stack exhaustion	Correct checksum
Required tail calls	Correct checksum	Correct checksum
Trampoline	Correct checksum	Correct checksum

The compiler was Apple Clang 21.0.0, targeting arm64 macOS, in C++2c mode with pedantic errors. The builds used `-O0` or `-O2`, disabled link-time optimization, and ran with a 1 MiB stack limit. The five successful runs produced the same checksum, checked against an iterative reference. A debugger run of the failing variant showed alternating handler frames and a fault while storing into a new stack frame. A separate negative test rejected a marked transfer with a pending nontrivial destructor.

This is a test of behavior across build modes on one target. It measures no speedup and does not substitute for rebuilding a production workload.

6 Three implementations provide experience

Implementation	Documented facility	Material limits
Clang	`clang::musttail`. Enforced even without optimization. [ClangMusttail]	Signature and calling-convention restrictions, trivial destruction, and early termination of caller-local lifetimes.
GCC 15	`gnu::musttail`. Required tail call or diagnostic. [GCC15] [GCCMusttail]	Target restrictions. Optimization can change which calls are accepted. Pending nontrivial destruction prevents a transfer.
MSVC Build Tools 14.50	`msvc::musttail`. Tail call or compilation error. [MSVCMusttail]	Experimental, x64 only, and requires `/O2` or `/O2 /GL`. Calling-convention and argument-space restrictions apply.

The mechanisms are implemented, but their accepted programs differ. Standardizing the spelling alone would not resolve those differences. Only the Clang mechanism was executed for this paper.

CPython 3.14 provides further use evidence through its optional tail-call interpreter. [PythonTailCalls] Its implementation report also records poor performance with `musttail` alone, without the selected calling convention. [PythonTailCallExperience] That experience prevents attributing the interpreter's performance to this facility alone. The Wind Waker HD recompilation project requires Clang's facility for generated game code. [WindWakerTailCalls] This establishes a game-related use, not general engine demand or a measured engine benefit.

7 The proposed contract

The starting domain is direct and function-pointer calls between non-variadic free functions or static member functions. Corresponding parameter types are matching scalar types, results are a matching scalar type or void, and calling conventions and exception specifications match. This deliberately starts below the full domain accepted by existing implementations.

For the initial domain, the intended contract is:

1. The marker applies to a return whose operand is a call. A void callee also uses a return statement.
2. After argument evaluation and before entering the callee, the lifetimes of the caller's automatic-storage variables and function parameters end. Their storage cannot be relied on by the callee. Objects with static or thread storage duration and objects owned outside that activation retain their ordinary lifetimes.
3. Reject a transfer with pending nontrivial destruction or an active exception handler that would require retaining the caller. No destructor is skipped to make a call eligible.

4. The transfer does not retain the caller's activation for a later return. Repeated marked transfers among a fixed set of eligible functions must not accumulate retained activations or equivalent per-transfer storage. Ordinary calls made within those functions retain their ordinary behavior.
5. Diagnose an unsupported transfer. If a build mode or target cannot provide the guarantee, rejection is required. An accepted transfer has the same guarantee regardless of optimization settings.

The desired consequence is stack use bounded independently of the number of marked transfers. This is a design contract, not proposed standard wording. Early lifetime termination can invalidate pointers that are valid across an ordinary call. Detection of all escaped local addresses is not assumed. The supported calling conventions, argument temporaries, exception propagation, and precise cleanup rules need implementation agreement.

8 What standardization would add

A loop with a switch already expresses bounded dispatch. A trampoline can retain separate handlers by having them return the next operation to an outer loop. Both are valid alternatives. The experiment confirms the trampoline alternative. They change the dispatch structure or handler interface. A library wrapper cannot impose a required tail transfer on a return statement inside an arbitrary function body.

A macro can already select a vendor attribute and reject a configuration without support. Standardization is therefore not necessary merely to make today's code safe. Its useful contribution would be common lifetime, cleanup, signature, and diagnostic rules, together with reliable feature detection and an agreed minimum supported domain. It would let libraries depend on that contract rather than infer it from compiler-specific facilities. It would not make unsupported targets capable or necessarily eliminate their fallbacks.

9 Two standardization objections need answers

9.1 Attributes normally have optional semantics

The current C++ attribute model describes optional semantics, [dcl.attr.grammar]. Vendor attributes do not establish that early lifetime termination fits that model. WG14's draft on generalized function calls explicitly chose `return goto` syntax because the operation changes observable behavior. [CGeneralizedCalls] The proposed `always_tail` spelling requires either a demonstrated fit within the attribute model or a deliberate change to it. This remains a design question, not an editorial naming issue.

9.2 WG21 rejected a nearby request for a stack guarantee

CWG2454 asked about tail recursion and coroutine symmetric transfer. Its July 2020 NAD rationale treated the question as one of implementation limits and objected to normatively prescribing implementation techniques. [CWG2454] It was not a decision on this proposal, but an unexplained promise of bounded stack use would face the same objection.

The direction proposed here is to specify a control transfer that ends the caller activation, with diagnosed rejection when unsupported, rather than require a particular machine instruction or numerical stack limit. WG14's N3582 provides precedent for describing that operation and permitting rejection of unsupported cases. It is a draft, not an adopted C++ facility. [CGeneralizedCalls] Whether this approach answers CWG's objection remains to be established.

10 Evidence required before a recommendation to adopt

The next revision needs:

1. A common supported domain demonstrated on more than one compiler and target, including indirect calls and separate translation units in debug and optimized builds. Rejection behavior must also be checked.

2. A mapping of a production user's actual signatures and lifetime requirements to that domain, with direct maintainer feedback on the proposed contract. Existing use of vendor facilities is not endorsement of this design.
3. Wording that resolves the attribute-model and implementation-limit objections, including argument temporaries, exception cleanup, constant evaluation, and instrumentation. Non-static member calls and aggregate arguments and results remain outside the starting domain. Virtual thunks need particular care because an intermediate thunk can retain a frame. [ClangMusttail]

Any performance justification additionally needs a controlled workload comparison with the existing loop or trampoline, holding the calling convention and other optimizations fixed. This paper makes no performance claim.

11 Conclusion

Protobuf supplies a concrete C++ use, three compiler families supply implementation experience, and the reduced test demonstrates why ordinary optimization is insufficient as a contract. These justify work on a common checked tail-transfer facility. They do not yet justify adopting this particular attribute. Feedback is requested on the transfer semantics, the minimum supported domain, and the two standardization objections.

12 Disclosure

The author provides information and serves at the pleasure of the committee. This draft requests design feedback. Local execution covers one compiler and target. Deployment evidence comes from project documentation and source, without rebuilding those projects.

13 References

[CGeneralizedCalls] N3582: C Extensions to support generalized function calls.
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3582.pdf

[ClangMusttail] Attributes in Clang: musttail.
https://clang.llvm.org/docs/AttributeReference.html#musttail

[CWG2454] CWG2454: Tail recursion and coroutine symmetric transfer.
https://wg21.link/CWG2454

[GCC15] GCC 15 Release Series: Changes, New Features, and Fixes.
https://gcc.gnu.org/gcc-15/changes.html

[GCCMusttail] GCC Common Attributes: musttail.
https://gcc.gnu.org/onlinedocs/gcc/Common-Attributes.html#index-musttail

[MSVCMusttail] Attributes in C++: msvc::musttail.
https://learn.microsoft.com/en-us/cpp/cpp/attributes?view=msvc-170#msvcmusttail

[ProtobufTailCalls] Protobuf C++ parser: TagDispatch, ToTagDispatch, and ParseLoop.
https://github.com/protocolbuffers/protobuf/blob/7eda095f12dcc486b6810a20dc86df49a136881a/src/google/protobuf/generated_message_tctable_impl.h#L1023-L1072

[PythonTailCallExperience] A new tail-calling interpreter for significantly better interpreter performance.
https://github.com/python/cpython/issues/128563

[PythonTailCalls] Python 3.14 configuration: –with-tail-call-interp.
https://docs.python.org/3.14/using/configure.html#cmdoption-with-tail-call-interp

[WindWakerTailCalls] The Legend of Zelda: The Wind Waker HD native port: Building.
https://github.com/ZeldaWWHDRecomp/ZeldaWWHDRecomp#building